

DataKern | AI Engineering Assignment

Session 2: LLM Internals, Observability, and Containerized Deployment

A guided engineering workflow for building, containerizing (Docker), and observing (Phoenix) a real-world AI Chatbot.

DATAKERN

ASSIGNMENT

www.data-kern.com

Before you begin: What are you actually doing?

Congratulations on your role as a Junior AI Engineer at **RiverBank**!

In Session 2, you opened the "black box" and learned that an LLM is not magic—it is a deterministic 9-step processing pipeline. You also learned that controlling the output requires understanding the sampling dials (**Temperature**, **Top-K**, and **Top-P**), and debugging system behavior requires observability (**Arize Phoenix**).

Now, your engineering lead has given you a real-world production challenge:

"The interactive chatbot works in your local terminal and browser, but 'it works on my machine' is not an acceptable standard for enterprise software. We need this application isolated locally using a virtual environment (`venv`), packaged into a reproducible `Docker` container, and deployed so that anyone on the team can run the chatbot UI and observe the flight data recorder in real time."

As an AI engineer, writing an LLM prompt is only the first step. The real engineering discipline involves: 1. **Local Environment Isolation (`venv`)**: Keeping project dependencies completely separated from your operating system. 2. **Production Containerization (`Docker`)**: Creating an isolated, portable environment where the application, OS libraries, network ports, and runtime are identical regardless of whether it runs on macOS, Windows, Linux, or in the cloud. 3. **Interactive UI & Real-Time Observability (`Streamlit` + `Phoenix`)**: Providing an intuitive user interface with full control over sampling dials (**Temperature** and **Top-P**) while maintaining a live "camera inside the box" to inspect latency, token costs, and internal traces.

What you will produce

- A structured project repository with an isolated virtual environment (`.venv`) and a pinned `requirements.txt` .
 - An interactive chatbot application in `src/app.py` combining **Streamlit**, **Groq**, and **Phoenix/OpenTelemetry** auto-instrumentation, complete with **Temperature** and **Top-P** sidebar controls.
 - A production `Dockerfile` and startup script (`entrypoint.sh`) that builds the container, exposes necessary ports, and runs both the Chatbot UI and Phoenix Observability server.
 - A trace analysis and architecture report in `docs/trace_analysis.md` capturing live evidence from your containerized chatbot across different sampling configurations.
 - Incremental, professional Git commits reflecting standard software engineering practices.
-

1. Mental Model: Local Development vs. Docker in Enterprise AI

The "Works on My Machine" Trap

When you write Python code on your laptop, it relies on a delicate web of local conditions: - The exact Python minor version installed on your machine. - Operating-system-specific C-compilers and system libraries (e.g., macOS clang vs. Windows MSVC vs. Linux GCC). - Open network ports and background services. - Local environment variables and file paths.

When you pass your code to a teammate or deploy it to an AWS/GCP cloud server, it frequently crashes. In enterprise AI engineering, relying on your laptop's unique setup is unacceptable.

Level 1: The Local Virtual Environment (`.venv`)

A **Python Virtual Environment** is a lightweight directory tree that contains a specific Python binary plus an isolated set of additional packages (`site-packages`).

- **What it does:** Prevents project A (which might need `pydantic v1`) from conflicting with project B (which needs `pydantic v2`) on your own computer.
 - **Its limitation:** It **only** isolates Python packages. It does **not** isolate the operating system, system-level C-libraries, or network configurations. A virtual environment created on a Mac cannot run on a Windows machine or a Linux server.
-

Level 2: Containerization with Docker (The Industry Standard)

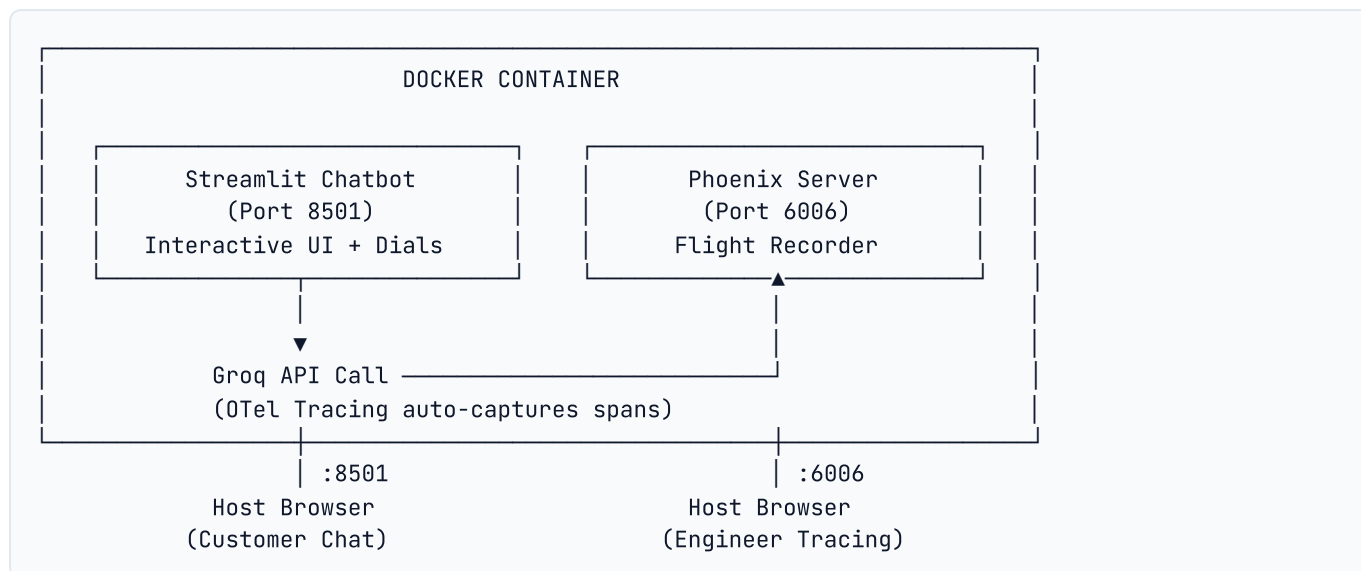
Docker solves the environment problem completely by packaging the application, the Python runtime, all system libraries, and the underlying Linux operating system into a single, standardized unit called a **Container**.

The Shipping Container Analogy

Before modern shipping containers were invented in the 1950s, cargo (barrels of wine, bags of grain, crates of machinery) was loaded loosely onto ships. Every port had different cranes, loading was agonizingly slow, and items regularly broke in transit.

The invention of the standardized **intermodal shipping container** revolutionized world trade: whether a container carries bananas or microchips, it fits on every ship, train, truck, and crane anywhere on earth.

Docker does the exact same thing for software: - Instead of shipping loose Python files and hoping the destination server has the right libraries, you seal the application into a standardized software container. - It runs identically on your local MacBook, your colleague's Windows laptop, and an enterprise Linux cluster in the cloud.



The Step-by-Step Mental Model: Dockerfile → Image → Container

To master Docker, you only need to understand three core concepts:

Concept	Mental Model	What it actually is
1. Dockerfile	The Recipe / Blueprint	A plain text file containing step-by-step instructions on how to build your environment (e.g., base OS, files to copy, packages to install).
2. Docker Image	The Frozen Snapshot / Cake	The compiled, read-only template built from your Dockerfile using <code>docker build</code> . It contains the OS, runtime, and dependencies frozen in time.
3. Docker Container	The Living Instance	The active, running process started from an image using <code>docker run</code> . You can start, stop, or spin up 10 identical containers from one image.
4. Port Forwarding (<code>-p</code>)	The Tunnel / Bridge	Containers run in an isolated network sandbox. Port forwarding (<code>-p 8501:8501</code>) opens a tunnel connecting port 8501 on your host machine to port 8501 inside the container.

2. Understanding Streamlit: Execution Model & Session State

Streamlit allows AI engineers to build interactive web interfaces in pure Python. However, it operates fundamentally differently from traditional web frameworks like Flask or Django.

The Streamlit Execution Mental Model: "Script Runs Top to Bottom"

- In traditional web development, a server stays running and only executes a specific function when a specific URL route is called.
- **In Streamlit, the ENTIRE Python script re-runs from line 1 to the end every time the user interacts with any widget** (types a message, adjusts a slider, clicks a button).

The State Problem & `st.session_state`

Because the entire script re-executes on every user action, **any ordinary Python variable will be wiped out and reset to its initial value on every turn!**

If you wrote:

```
messages = [] # ❌ Wiped out every time the user types!
```

Your chatbot would suffer from immediate amnesia after every single message.

The Solution: `st.session_state` is a dictionary-like persistent storage object provided by Streamlit that **survives script re-runs**. It stays preserved in memory for that specific browser session:

```
# Initialize persistent chat history only on the very first run
if "messages" not in st.session_state:
    st.session_state.messages = [
        {"role": "system", "content": SYSTEM_PROMPT}
    ]
```

Essential Streamlit Chat Commands Explained

Command	Purpose	How it works
<code>st.title("...")</code>	Page Heading	Renders a styled main title at the top of the interface.
<code>st.sidebar.slider(...)</code>	Sidebar Controls	Places interactive sliders in the left sidebar to adjust parameters (e.g., Temperature, Top-P) without cluttering the chat area.
<code>with st.chat_message(role):</code>	Message Container	Creates a native chat bubble with appropriate styling and avatars for "user" or "assistant" .
<code>st.markdown(text)</code>	Formatted Text	Renders markdown formatting (bold, code blocks, lists) inside a chat bubble.
<code>if prompt := st.chat_input("..."):</code>	Input Box + Walrus Operator	Renders a fixed bottom chat bar. The Python "walrus operator" (<code>:=</code>) assigns user text to <code>prompt</code> and evaluates truthiness in a single line.

3. The Three Sampling Dials: Temperature, Top-K, and Top-P

In Slide 9B of Session 2, we saw that the LM Head (Step 6) outputs a probability distribution across 50,000+ vocabulary tokens. Before one token is selected, three dials shape and filter that distribution:

Full Probability List (50,000+ tokens)

↓

① TEMPERATURE → Reshapes the curve (sharp peak vs. flat spread)

↓

② TOP-K → Hard cut: keeps only the top K highest-probability words

↓

③ TOP-P → Dynamic cut (Nucleus): keeps smallest set summing to P%

↓

Final Token Picked & Appended to Prompt (Step 8 Autoregressive Loop)

1. Temperature (\$T\$)

- **What it does:** Divides the model's raw logits by \$T\$ before computing the Softmax probability.
- **Low Temperature (\$0.1\$):** Squeezes the distribution. The top candidate becomes overwhelmingly dominant. Output is deterministic, focused, and safe.
- **High Temperature (\$1.5\$):** Flattens the distribution. Secondary and tertiary words gain realistic selection chances. Output is creative, diverse, but prone to wandering.

2. Top-K Sampling

- **What it does:** Restricts the candidate pool to a **fixed integer count** of the top \$K\$ tokens.
- **Example:** If \$K=50\$, only the 50 highest-probability words are eligible. The other 49,950 tokens are discarded, preventing the model from picking completely nonsensical words even at high temperatures.

3. Top-P Sampling (Nucleus Sampling)

- **What it does:** Restricts the candidate pool dynamically based on **cumulative probability mass**.
- **Example (\$P=0.9\$):** Sort all tokens by probability and sum them until they reach 90%. Only those tokens are eligible.
- If the model is extremely confident, 2 tokens might account for 90% of the mass \$\to\$ only those 2 tokens are considered.
- If the model is uncertain, 40 tokens might be needed to reach 90% \$\to\$ the candidate pool expands dynamically.
- **Why it matters:** Unlike Top-K (which is rigid), Top-P adapts to the context.

Tools and Project Setup

1. Create your project folder

Create a new folder named `riverbank-chat-service`, open it in VS Code, and set up the following directory layout:

```
riverbank-chat-service/
├── src/
│   └── app.py
├── docs/
│   └── trace_analysis.md
├── Dockerfile
├── entrypoint.sh
├── .dockerignore
├── .gitignore
├── requirements.txt
└── README.md
```

2. Create and activate a Python virtual environment

Open VS Code's Terminal (**Terminal** → **New Terminal**). Ensure the terminal is inside your project root folder, then run:

```
python -m venv .venv
```

Activate the virtual environment: - **Windows (Command Prompt):** `.venv\Scripts\activate` - **Windows (PowerShell):** `.venv\Scripts\Activate.ps1` - **macOS / Linux:** `source .venv/bin/activate`

When activated, you will see `(.venv)` at the beginning of your terminal prompt.

3. Install required libraries & freeze dependencies

With the virtual environment activated, install the required libraries:

```
python -m pip install --upgrade pip
python -m pip install groq streamlit arize-Phoenix opentelemetry-sdk opentelemetry-exporter-otlp o
```

Save your exact installed versions to `requirements.txt` :

```
python -m pip freeze > requirements.txt
```

Create a `.gitignore` file and add:

```
.venv/
__pycache__/
*.pyc
.env
```

Important: Never commit `.venv/` to Git. Virtual environments contain binaries specific to your local machine and should never be shared or pushed to a repository.

4. Configure your Groq API Key

Set your `GROQ_API_KEY` in your terminal: - **macOS / Linux:** `export GROQ_API_KEY="gsk_your_actual_key_here"` - **Windows (Command Prompt):** `set GROQ_API_KEY=gsk_your_actual_key_here` - **Windows (PowerShell):** `$env:GROQ_API_KEY="gsk_your_actual_key_here"`

Part A — Implement: The Observed Streamlit Chatbot with Sampling Dials

Goal: Build an interactive chat application in `src/app.py` that connects Streamlit, Groq, Phoenix OpenTelemetry instrumentation, and interactive controls for both Temperature and Top-P.

5. Architectural Blueprint & Starter Scaffold for `src/app.py`

Do not copy-paste a finished script. Use this scaffold as your engineering guide, reading each hint and implementing the `TODO` sections:

DATAKERN

```

import os
import streamlit as st
from groq import Groq

# -----
# HINT 1: Observability Imports
# You need `register` from `phoenix.otel` and `GroqInstrumentor` from
# `openinference.instrumentation.groq`.
# -----
from phoenix.otel import register
from openinference.instrumentation.groq import GroqInstrumentor

# TODO 1: Initialize Phoenix OpenTelemetry Tracer
# Method to use: register(project_name="...", endpoint="http://127.0.0.1:6006/v1/traces")
# Then call: GroqInstrumentor().instrument(tracer_provider=tracer_provider)
# → Write your code below:

# -----
# HINT 2: Environment Guard & Client Setup
# Check `os.environ.get("GROQ_API_KEY")`. If missing, use `st.error(...)`
# and `st.stop()`.
# Instantiate the Groq client: Groq(api_key=...)
# -----
MODEL = "llama-3.3-70b-versatile"
SYSTEM_PROMPT = (
    "You are a professional banking assistant for RiverBank. "
    "Provide accurate, helpful, and concise customer support."
)

st.title("🏦 RiverBank Customer Assistant")

# -----
# HINT 3: Sidebar Controls (The Sampling Dials)
# Use `st.sidebar.slider(...)` to create two dials:
# 1. Temperature: range 0.0 to 2.0, default 0.2, step 0.1
# 2. Top-P: range 0.05 to 1.0, default 0.9, step 0.05
# -----
st.sidebar.header("🎛️ Sampling Dials")

# TODO 2: Create the temperature and top_p sliders
# → Write your code below:

# -----
# HINT 4: Session State Management
# Check if "messages" is NOT in st.session_state.
# If not, initialize it as a list containing a dict for the system prompt:
# [{"role": "system", "content": SYSTEM_PROMPT}]
# -----

# TODO 3: Initialize st.session_state.messages
# → Write your code below:

# TODO 4: Loop through st.session_state.messages and render previous turns.
# (Hint: Skip the message if its role is "system". For other roles, use:
# with st.chat_message(msg["role"]): st.markdown(msg["content"]))

```

```
# → Write your code below:

# -----
# HINT 5: Chat Input & API Execution
# Use `if prompt := st.chat_input("..."):`
# Inside the block:
# 1. Immediately render the user's prompt using with st.chat_message("user"): ...
# 2. Append the user message dict to st.session_state.messages
# 3. Call client.chat.completions.create() passing:
#     model=MODEL, messages=st.session_state.messages,
#     temperature=temperature, top_p=top_p
# 4. Extract reply: response.choices[0].message.content
# 5. Render reply in assistant bubble and append to st.session_state.messages
# -----

# TODO 5: Implement the chat input and API call flow
# → Write your code below:
```

6. Verify Locally

Before containerizing, verify that your script works on your local machine: 1. In one terminal, start Phoenix: `bash python -m phoenix.server.main serve` 2. In a second terminal (with `.venv` activated), start Streamlit: `bash streamlit run src/app.py` 3. Open `http://localhost:8501`, send a test message, and confirm the trace appears at `http://localhost:6006` 4. Stop both processes (`Ctrl+C`) once verified.

Part B — Containerize: Production Docker Packaging

Goal: Package the entire application stack into a clean, reproducible Docker image and deploy it.

7. Create `entrypoint.sh`

In production, a container typically runs a single process. However, here we need two services to communicate: 1. The **Phoenix Collector & Server** on Port `6006` . 2. The **Streamlit Chatbot** on Port `8501` .

Create `entrypoint.sh` using the following architectural hints: - Start with the shebang `#!/bin/bash` and `set -e` (exits immediately if any command fails). - Launch Phoenix in the background using `python -m phoenix.server.main serve &` (the `&` operator is critical—it runs the server in the background so the script does not block). - Add a brief pause (`sleep 3`) to allow Phoenix's HTTP listener to bind to port 6006. - Launch Streamlit in the foreground with production flags: - `--server.port=8501` - `--server.address=0.0.0.0` (Essential: binds to all container network interfaces so outside traffic can reach it) - `--server.headless=true` (Disables browser-opening attempts inside a headless Linux container) - `browser.gatherUsageStats=false`

Make the script executable on Unix systems:

```
chmod +x entrypoint.sh
```

8. Create `.dockerignore`

Prevent local virtual environments, git history, and cache files from bloating the Docker image:

```
.env
.git
__pycache__
*.pyc
.env
docs/
```

9. Create `Dockerfile`

Create your `Dockerfile` using the following directives and rationale:

```
# 1. Base Image: Choose a lightweight official Python runtime
FROM python:3.11-slim

# 2. Working Directory: Set the internal app path
WORKDIR /app

# 3. Environment Variables:
# - PYTHONDONTWRITEBYTECODE=1: Prevents Python from writing .pyc files inside container
# - PYTHONUNBUFFERED=1: Flushes stdout/stderr immediately for real-time terminal logs
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# 4. Dependency Layer Caching Strategy:
# HINT: Copy requirements.txt FIRST before copying any application code!
COPY requirements.txt .

# 5. Install Dependencies:
# HINT: Use --no-cache-dir to keep the Docker image slim
RUN pip install --no-cache-dir -r requirements.txt

# 6. Copy Application Source & Entrypoint:
COPY src/ ./src/
COPY entrypoint.sh .

# 7. Document Exposed Ports:
# Both Streamlit (8501) and Phoenix (6006) must be exposed
EXPOSE 8501 6006

# 8. Container Entrypoint:
ENTRYPOINT ["/entrypoint.sh"]
```

Pause and Observe: Why did we copy `requirements.txt` and run `pip install` *before* copying `src/` ? In Docker's caching engine, each line is an immutable layer. If you modify application code in `src/app.py` , Docker reuses the cached layer containing all your installed packages instead of re-downloading them every time!

Part C — Deploy & Run: The Containerized System

Goal: Build the Docker image, run the container, forward ports, and verify live execution.

10. Build the Docker Image

From your project root, run:

```
docker build -t riverbank-chat:latest .
```

11. Run the Container

Start the container with port forwarding for both 8501 and 6006, passing in your GROQ_API_KEY :

- **macOS / Linux:** `bash docker run -p 8501:8501 -p 6006:6006 -e GROQ_API_KEY="$GROQ_API_KEY" --name riverbank-app riverbank-chat:latest`
- **Windows (PowerShell):** `powershell docker run -p 8501:8501 -p 6006:6006 -e GROQ_API_KEY=$env:GROQ_API_KEY --name riverbank-app riverbank-chat:latest`

Part D — Validate & Observe: The Flight Recorder in Action

Goal: Interact with the deployed container, execute diagnostic scenarios, and document how the 9 internal LLM steps behave in production.

Open two browser tabs side-by-side: 1. **Chatbot UI:** `http://localhost:8501` 2. **Phoenix Observability:** `http://localhost:6006`

Execute the following 4 diagnostic scenarios in the Chatbot UI:

Scenario 1: Operational Conciseness (Low Temperature, Standard Top-P)

- **Settings:** Temperature = 0.1, Top-P = 0.9
- **Prompt:** "What are the standard branch hours and deposit limits for RiverBank personal accounts?"
- Note the response quality, token count, and latency in Phoenix.

Scenario 2: Token Explosion & Autoregressive Latency (Step 8)

- **Settings:** Temperature = 0.1, Top-P = 0.9
- **Prompt:** "Provide an exhaustive 500-word analysis on the economic history of fractional-reserve banking from renaissance Italy to contemporary central banks."
- Observe the noticeable latency increase before the full response finishes generating.

Scenario 3: The Sampling Dial Experiment (Comparing Top-P)

- **Run A (Narrow Nucleus):** Set Temperature = 1.0, Top-P = 0.1.
- **Prompt:** "Describe the color of money in one creative sentence."

- Send message. Observe response.
- **Run B (Wide Nucleus):** Keep `Temperature = 1.0` , but increase `Top-P = 0.95` .
- **Prompt:** `"Describe the color of money in one creative sentence."` (Exact same prompt).
- Send message. Compare how the vocabulary choice broadened between Run A and Run B.

Scenario 4: The Hallucination Probe (Step 4 & Step 5)

- **Settings:** `Temperature = 0.7` , `Top-P = 0.9`
- **Prompt:** `"What was the exact settlement amount paid by RiverBank in the 2023 Zurich vault dispute, and which arbitrator signed off on the agreement?"`
- Check Phoenix: Does the model invent details? Connect this to the absence of grounding context for Self-Attention (Step 4).

12. Document your analysis in `docs/trace_analysis.md`

Create `docs/trace_analysis.md` and complete the following report:

Section 1: Containerized Telemetry Audit

Scenario	Temp	Top-P	Prompt Tokens	Completion Tokens	Latency (ms / s)	Observation
1. Concise FAQ	0.1	0.9				
2. History Analysis	0.1	0.9				
3A. Creative (Top-P 0.1)	1.0	0.1				
3B. Creative (Top-P 0.95)	1.0	0.95				
4. Hallucination Probe	0.7	0.9				

Section 2: Architectural & Engineering Reflection

Answer the following questions based on your build and observations:

1. **Autoregressive Loop Mechanics (Step 8):**
2. In Scenario 2, why did latency increase significantly compared to Scenario 1 even though Groq's LPUs are extremely fast?
3. How does the number of completion tokens directly dictate the number of passes through the Transformer stack?
4. **The Sampling Dials (Step 7 — Temperature vs Top-P):**
5. In Scenario 3, explain what happened when `Top-P` was set to `0.1` versus `0.95` while Temperature was held at `1.0` .
6. Why is Top-P considered "adaptive" compared to Top-K?

7. Streamlit Session State:

8. Why would an ordinary Python variable (e.g. `history = []`) fail to maintain chat history in a Streamlit application?
9. What happens under the hood when a user types a message into `st.chat_input`?

10. Packaging & Deployment with Docker:

11. What is the difference between a `Dockerfile`, an `image`, and a `container`?
12. Why do we use `.venv` during local development, but install directly into the container's environment inside the `Dockerfile`?

Git Checkpoints and Submission

Commit your project in logical, clean increments:

```
git status
git add requirements.txt Dockerfile entrypoint.sh .dockerignore .gitignore src/ docs/
git commit -m "feat: implement streamlit chat with groq, phoenix tracing, and sampling sliders"
git commit -m "build: containerize application with multi-port dockerfile"
git commit -m "docs: complete containerized trace analysis report"
git push
```

A note from Uma Kiran

A prototype on a laptop is a toy; a reproducible, containerized, and observable service is software engineering. As AI systems become central to enterprise architecture, the engineers who stand out are those who understand what happens inside the model, how to package applications reliably, and how to verify their system's behavior with real data.

Important learning rule: Do not use AI code-generation or coding-agent tools to produce your assignment solution. This includes ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar tools. You may use official documentation and search for errors, but you must write and explain your own solution.

Don't focus on getting the answer. Focus on learning how to engineer the solution.

— Uma Kiran | DataKern | www.data-kern.com